Deployability, Portability, and Containers

Chapter 9 — Software Architecture (Pautasso)

From **design-time** qualities to **production** — and how we ship continuously

Chapter contents (Pautasso)

- **Deployability metrics** — latency, throughput, time, cost
- **To change or not to change** — risk, releases, reversibility
- **CI, CD, continuous deployment** — software production pipeline
- **Platform independence** — virtualization, containers, portability

Learning objectives

- **Measure** deployability (latency, throughput, downtime, cost)
- **Contrast** traditional vs continuous delivery; **CI / CD / CD** stages
- **Explain** release strategies (blue/green, canary, shadow, …) and **testing** layers
- **Portability** — config, paths, images; **containers** vs VMs
- **Operate** — health probes, rollbacks, immutable artifacts (**CityBite** examples)

Recap — Lecture 8

Compatibility and Coupling

- Coupling strength; breaking vs non-breaking change; semver & coexistence
- → Today: **packaging**, **pipelines**, and **runtime environments**

Recap → Chapter 9

- Lecture 8: **contracts** at boundaries
- Lecture 9: **how often** we ship, **how** we build/test/package, **where** binaries run
- Next lectures (course arc): **scalability, availability, flexibility** — runtime qualities

From design to production

- So far: architecture qualities mostly at **design time**
- This lecture: **deploy** the system — **deployability** and **portability**
- Before runtime scale/failover topics: we need **repeatable** execution in **containers** / clouds

The age of continuity

- Past: major releases every **years** (e.g. early Windows cadence)
- Today: some providers ship **many times per second** — “end of the release” from the user’s view
- **Deployability** = enabling **continuous** improvement (features, fixes) **with confidence**

Deployability metrics (1)

Latency — how long until effect?

- Dev writes code → user sees feature or bugfix in **production**
- User feedback → routed decision → fix → user **accepts** improvement
- One-click deploy vs **multi-month** project per cycle?

Deployability metrics (2)

Throughput — how many releases per hour/day/month?

Time / disruption

- How long is the system **unavailable** during upgrade?
- Hit **marketing** window (e.g. holiday launch)?

Cost

- How many **people** per release? How much **build/test** infrastructure (VMs, parallel tests)?

Traditional vs continuous (summary)

	Traditional	Continuous
Latency	High	Small
Throughput	~1 / years	Many / day
Downtime	Noticeable	Aim for **zero** perceived
Deadline	“Death march” risk	No single big-bang window
People	Release engineering team	**Automation** (goal: minimal toil)
Pipeline	Ad hoc, manual	**Industrial**, repeatable

Release: opportunity and risk (Nygard)

Opportunity — features, fixes, performance, revenue, retention, beat competition

Risk — new failure modes, new bugs, capacity cost, support/training load, **failed deploy**, unrecoverable states

Architecture + process should **maximize upside** and **bound downside**

“No change = no risk” — the trap

- Only **unused** software never needs to change
- **Minimize risk *of* change** — not “never change”
- **Small**, **frequent** releases + **reversible** rollouts (like **atomic** state transitions: commit / rollback)

Big Bang vs continuous

- **Waterfall handoff**: build → throw over wall → ops runs; painful transition → “if it hurts, do it **more often**”
- **Continuous**: dev and ops **in parallel** — tight feedback; failed builds/releases are **normal** events you recover from

Speed vs quality — automation breaks the tradeoff

- Old view: fast ⇒ low quality; high quality ⇒ slow
- **Continuous** delivery needs **both**: trust comes from **automated** tests + repeatable packaging
- Pipeline **is software** too — version and test your Jenkins/GitHub Actions/K8s manifests

Software production pipeline (idea)

Input: source **Output:** immutable **artifact** ready for prod

- Build ≠ only compile — **unit tests**, static analysis, docs generation
- **Integration** — assemble components; integration tests
- **Capacity / performance** — compare to baseline; **acceptance** / UAT
- **Decision:** ship or not → **deploy** → **smoke** in prod

Continuous integration (CI)

- Frequent **commit / merge**; automated **build + tests** (e.g. daily minimum)
- **Green mainline** — always have an **integrable**, runnable system after stages you automate

Continuous delivery (CD)

- Pipeline produces a **fully tested package ready** for production
- **Human** (or policy) **approval** before **deploy** — gate stays manual

Continuous deployment

- **Automatic** deploy to prod **if** all automated checks pass
- Requires very high **trust** in tests; consumers (e.g. **mobile app stores**) may still add delay

High quality at high speed (practices)

- Everyone can **see** the pipeline; **don’t push** on broken build; **watch** until green
- **Small** commits; **fast** early feedback; **nightly** heavier tests if needed
- **Reproducible** builds — **version dependencies** (snapshots), **immutable** release images

Types of testing (overview)

Type	Focus
Unit	Components; mocks for neighbors
Integration	System with real dependencies wired
User / E2E	UI scenarios — often partly manual
Capacity / perf	Resources vs targets (SLAs, cost)
Acceptance	Meets agreed criteria — release decision
Smoke	Prod / staging sanity after deploy

Testing in production always happens; pre-release tests reduce how often users are the first testers.

Release strategies

- **Big Bang / plunge** — everyone switches; hard rollback
- **Blue / green** — instant switch; old stack kept for **rollback** (cost: 2× capacity)
- **Shadow / dark launch** — new version gets **real input**; output not shown; compare
- **Pilot** — small trusted user set
- **Gradual / canary** — ramp traffic; **A/B** — pick winner by metrics

Virtualization (why deployability improved)

- **Hypervisor** decouples **guest OS + apps** from **physical hardware**
- **Elasticity** — resize VMs; **migration** — move workloads between hosts
- Cost: emulation gap vs bare metal; **isolation** vs **sharing** tradeoffs

Containers = lightweight isolation

- Same **host kernel**; **namespaces / cgroups** isolate processes
- Smaller footprint, **sub-second** start vs full VM boot
- **Security**: VM isolation generally **stronger**; containers improve **density** and **dev/prod parity**

VM vs container (comparison)

	VM	Container
Footprint	Often **GB**	Often **MB**+app
Boot	**Minutes** typical	**Seconds**
Guest OS	Per VM	**Shared** host OS
Isolation	Hardware-assisted	Namespace OS sandboxing
Persistence	Often **stateful** image	Treat as **stateless**; data outside

“Virtual hardware” = software

- **CPU/RAM/disk/network** allocation as **versioned config** (Terraform, K8s manifests)
- **Never mix** prod and test environments in config
- Same lesson as code: **review**, **test**, **rollback** infra changes

CityBite — deployability in the field

- **Anti-pattern:** SSH snowflakes, downtime on restart, dinner-rush fear of deploys
- **Target:** CI → **immutable image** → rolling update, **readiness** gates traffic
- Examples: `example1_*` (env, `PORT`, stdout logs), `example2_*` (`DATA_DIR` vs `./uploads`)

Portability (reminder)

- **Config** in environment; **secrets** injected — not baked into layers
- **Paths** — no `C:\…` or cwd-only writes; **reproducible** dependency snapshots

Twelve-factor (selected)

- **Logs** → stdout/stderr · **PORT** binding · **Disposability** / SIGTERM
- **Immutability** — don’t hot-edit prod images; rebuild and redeploy

Health probes (ops contract)

- **Liveness** vs **readiness** — avoid flapping and black holes during rollout

Assignment — CityBite on Kubernetes

See `ASSIGNMENT.md` — design + diagrams; **120 pts**

Submission

- **PR →** `lecture-9/submissions/YOUR_NAME/`

Takeaways

1. **Deployability is measurable** — latency, throughput, downtime, cost
2. **Continuous** flow needs **automation**, **reversibility**, and **culture** (DevOps)
3. **Virtualization & containers** improve **portability** and **replaceability**
4. **Release strategy** is an **architectural** choice (blue/green, canary, ...)
5. **Pipeline + infra as code** are **products** — version them like application code

References (readings)

- Jez Humble & David Farley — *Continuous Delivery* (2010)
- Len Bass, Ingo Weber, Liming Zhu — *DevOps: A Software Architect's Perspective* (2015)
- Michael T. Nygard — *Release It!*
- Gene Kim et al. — *The Phoenix Project*
- Twelve-Factor App, OCI — online

Questions?

Chapter PDF: `09_Deployability_Portability_and_Containers.pdf`